

FPGA I²C Address Translator

Project Documentation Author: Chaitanya Sabnis

1. Architecture Overview

Instead of trying to perform risky, on-the-fly bit manipulation on a single bus, I designed a Store-and-Forward architecture. The FPGA physically separates the I²C interface into two distinct domains: a Host Bus and a Sensor Bus.

I built an I²C Slave to interface with the host master, and an independent I²C Master to talk to the target sensor. A top-level wrapper acts as the bridge and data buffer between them. This approach completely eliminates bus contention and safely allows the target device to coexist with other devices that share its factory-default address.

2. FSM and Logic Breakdown

I kept the design modular using standard synchronous design practices. It breaks down into three main hardware blocks:

- **Top Wrapper (i2c_translator_top):** This acts as the traffic cop. It instantiates the Master and Slave modules, handles data buffering, and manages the triggers crossing between the two interfaces.
- **Host-Facing Slave FSM (i2c_slave):** Running on the main 100MHz clock, this module uses edge detection (scl_rise, scl_fall) to reliably sniff the bus. The FSM steps through standard I²C sequences (IDLE → RX_ADDR → ACK_ADDR, etc.). I also added asynchronous START/STOP detection so the state machine won't freeze up if the bus terminates unexpectedly.
- **Sensor-Facing Master FSM (i2c_master):** This is driven by a divided 100kHz clock (CLK_HALF_PERIOD = 500). Once the top wrapper gives it the green light, it autonomously drives the START condition, address, payload, and a clean STOP condition down to the physical sensor.

3. How the Address Translation Works

The actual translation relies on address filtering and forwarding.

- **Filtering:** The physical target sensor is hardwired to 0x48. I set up the FPGA Slave with a virtual address of 0x49. If the host master calls out to 0x48, the FPGA simply ignores it. This allows legacy devices on the host bus to communicate without any interference.
- **Capture:** When the host master transmits to 0x49, the FPGA Slave wakes up, pulls SDA low to ACK, and grabs the 8-bit payload.
- **Forwarding:** Once the payload is captured, the top wrapper triggers the FPGA Master. The Master (which is hard-coded to 0x48) starts a new transaction on the isolated Sensor Bus and pushes the data down to the physical device.

4. Design Challenges & Solutions

Challenge 1: Proving the devices can actually coexist

The core objective was ensuring two devices with the 0x48 address wouldn't conflict. I couldn't just test the RTL in isolation to prove this.

- **Solution:** I wrote a behavioral mock_i2c_device module for the testbench. By putting one legacy device on the host bus and one target device on the sensor bus, I was able to explicitly prove in the waveform that normal 0x48 traffic completely bypasses the FPGA, while 0x49 traffic gets caught and translated.

Challenge 2: Preventing data loss during translation

Buffering and forwarding data takes time. If the host master tries to read or write the next byte before the FPGA is done forwarding the first one, data will get dropped.

- **Solution:** I implemented standard I²C Clock Stretching (stretch_scl). The WAIT_FETCH state in the Slave FSM actively pulls the host's SCL line LOW, physically hitting "pause" on the host master until the FPGA Master finishes its job downstream.

Challenge 3: Nailing the I²C STOP condition timing

Generating a valid I²C STOP condition is tricky because SDA has to transition from low to high *while* SCL is held high.

- **Solution:** Instead of using messy combinational logic, I used the Master FSM's bit_cnt as a mini-sequencer at the end of the transaction (steps 8, 9, and 10). This guaranteed the SCL and SDA lines toggled in the exact sequence required to meet the physical protocol specs.